

EXERCISE ICC-02

A production-shaped Kubernetes Deployment

A manifest that works is easy. This one has to be one you would be willing to run in front of users.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

EXERCISE

icc-ex02-kubernetes-manifest

FORMAT

A single YAML file

SUBMIT AS

<ROLLNO>.yaml

TOPIC

Deployments, Services, probes, resources

MARKS

12

OFFERING

Odd Semester 2026-27

What to write

One YAML file containing a **Deployment** and a **Service** for a web application, separated by `---`.

The application image is `ghcr.io/example/web:1.4.2` and it listens on port 8080.

The checklist

Each item is checked mechanically, and the checker tells you which one failed.

The Deployment

- `apiVersion: apps/v1`, `kind: Deployment`
- **3 replicas**
- a `selector.matchLabels` that actually matches the pod template's labels
- one container using the image above, on `containerPort: 8080`

PITFALL · THE SELECTOR MUST MATCH THE TEMPLATE LABELS

They are two separate places in the file and nothing stops you writing different values. If they disagree, the Deployment manages **no pods**, reports no error, and your application never starts. The checker compares them.

COMMON MISCONCEPTION · `latest` IS A VERSION

It is a mutable pointer. Two nodes can pull it at different moments and end up running different code under the same name, and there is nothing to roll back to. The image tag must be a real version, and the checker rejects `latest`.

The probes

Both `livenessProbe` and `readinessProbe`, HTTP GET on port 8080, on **different paths**.

PROBE	QUESTION	CONSEQUENCE OF FAILURE
liveness	Is the process alive?	The container is restarted
readiness	Can it serve traffic right now?	Removed from the Service, not restarted

PITFALL · WHY THE SAME PATH FOR BOTH IS A REAL PRODUCTION INCIDENT

Suppose your readiness endpoint checks the database, which is correct, and you reuse it for liveness. The database has a thirty-second blip. Readiness correctly takes every pod out of the Service. Liveness then **restarts every pod simultaneously**, so when the database recovers there is nothing warm left to serve, and a thirty-second blip has become a multi-minute outage.

Liveness should test the process. Readiness may test dependencies.

Resources

Both `requests` and `limits`, for both `cpu` and `memory`.

DEFINITION · WHY REQUESTS ARE NOT OPTIONAL

`requests` is what the **scheduler reserves**; it decides which node you land on. It is also what the horizontal autoscaler measures against, since it scales on a percentage *of the request*.

With no requests the scheduler assumes the pod needs nothing, packs the node, and everything on it degrades together. This is the most common cause of a mysteriously unhealthy cluster.

Security

- `runAsNonRoot: true`, on the pod or the container
- **no plaintext credential** anywhere in the file

For the secret, use a reference rather than a value:

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: web-db
        key: password
```

The Service

- selects the Deployment's pods by label
- `port: 80` forwarding to `targetPort: 8080`

Checking your work

```
./selfcheck.sh
```

The checker runs in four stages and reports each separately, so you can see exactly which requirement is unmet.

NOTE · SCHEMA VALIDATION NEEDS A CLUSTER

`kubectl` contacts the API server even for a client-side dry run, because it has to map a kind to a resource. With no cluster running, the checker skips that step and says so; the structural checks still run. To get full schema validation, start a local cluster first:

```
kind create cluster
```

How this is marked

STAGE	MARKS	WHAT IT CHECKS
Valid YAML and Kubernetes objects	2	Parses, every document has <code>apiVersion</code> and <code>kind</code>
Deployment correctly specified	4	Replicas, matching selector, pinned image, port
Service selects the pods	2	Selector matches, port mapping correct
Production checklist	4	Distinct probe paths, requests and limits, non-root, no plaintext secret

Submitting

AIE23001.yaml

One file, both documents.